

Movie Knowledge Graph

A Linked Open Data Application

Do Tuan Hoang (HUST)

Hoang.DT261219M@sis.hust.edu.vn

Le Thi Nguyet Nga (HUST)

Nga.LTN261066M@sis.hust.edu.vn

Nguyen Truong Huy (HUST)

Huy.NT261260M@sis.hust.edu.vn

Nguyen Tien Khuong (HUST)

Khuong.NT261220M@sis.hust.edu.vn

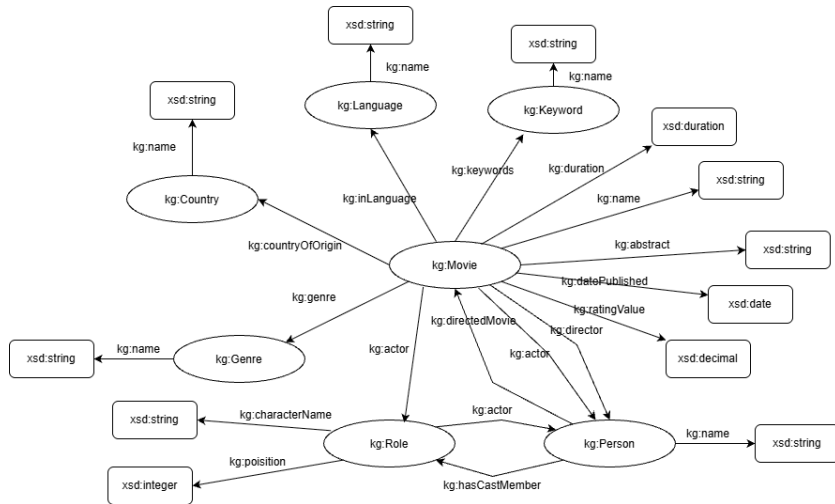
October 10, 2026

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Define an ontology: overview



Define an ontology: classes and instances

All domain terms use `kg: = https://example.org/ontology/`.

- **Movie:** Iron Man 2, identified by TMDb ID 10138.
- **Person:** a shared actor or director resource.
- **Role:** one acting participation, with character and cast order.
- **Genre:** Drama, Action, and other genre instances.
- **Country:** countries associated with movie production.
- **Language:** original and spoken languages.
- **Keyword:** descriptive terms shared by movies.

A class is a category; an instance is a particular member of that category.

Define an ontology: properties and literals

Object properties connect resources

- Movie relationships: actor, director, genre, keywords, countryOfOrigin, inLanguage.
- Role to person: actor; movie to webpage: referencePage.
- Reasoning relationships: directedMovie, hasCastMember.

Datatype properties connect resources to literal values

- name, abstract, characterName: strings.
- datePublished: date; duration: duration.
- position: integer; ratingValue: decimal.

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Collect relevant data in the movie domain

- **Original data:** TMDB 5000 Movie Dataset on Kaggle. Movie metadata and credits in two CSV files.
- **Processed data:**
<https://github.com/tomodachii/semantic-web/tree/main/data/prepared>. 100 movies, up to ten cast members per movie, directors only; eleven CSV tables.
- **External data:** Wikidata and DBpedia for linking local resources.

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Transform collected data: stable identifiers

Each entity receives a URI based on its type and source ID.

Movie ID 10138:

<https://example.org/movie/10138>

Person ID 3223:

<https://example.org/person/3223>

Genre ID 28:

<https://example.org/genre/28>

The same person URI is reused across movies and credits. Titles are display values, while source IDs identify resources.

Transform collected data: RDF triples

A movie becomes a resource with relationships and typed literals [2].

```
@prefix kg: <https://example.org/ontology/> .
```

```
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
<https://example.org/movie/10138>
```

```
  a kg:Movie ;
```

```
  kg:name "Iron Man 2" ;
```

```
  kg:datePublished "2010-04-28"^^xsd:date ;
```

```
  kg:genre <https://example.org/genre/28> .
```

Output: 100 movies and 11,603 triples in movies.ttl.

This implements URI-based RDF. Public deployment still needs persistent, resolvable URIs in place of example.org.

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Establish links: matching strategy

A script finds links automatically by querying Wikidata with SPARQL.

- **Movie:** TMDB ID → title + year → manual check
- **Person:** TMDB ID → name in cast/director of a linked film → manual check
- **Country:** ISO 3166-1 code
- **Language:** ISO 639-1 code
- **Genre:** film-genre label + synonym → manual check
 - e.g. “Action” → “action film”, “Animation” → “animated film”

IDs are exact, so they come first; names are ambiguous.

Unmatched or ambiguous cases are verified by hand on Wikidata.

Establish links: represent with owl:sameAs

Links connect our resources to other datasets [1].

```
<.../movie/10138>
```

```
  owl:sameAs <http://www.wikidata.org/entity/Q205028> ,  
              <http://dbpedia.org/resource/Iron_Man_2> .
```

```
<.../genre/28>
```

```
  skos:closeMatch <http://www.wikidata.org/entity/Q188473> .
```

- Genres use the weaker `skos:closeMatch`.
- 99 movies, 805 people, and 56 lookup resources approved.
- **1,901** link triples in total.

External links provide the cross-dataset connection for the fifth star.

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Interface via SPARQL endpoint/terminal to query data

Our terminal interface loads movie data, the ontology, and optional link files. SPARQL selects matching graph patterns [3].

```
python scripts/ask.py -q queries/4.sparql
```

```
python scripts/ask.py -q queries/11.sparql --reasoning
```

```
python scripts/ask.py -q queries/show_links.sparql  
--links output/movie_links.ttl
```

The last example is one command, wrapped here for display. `-reasoning` adds inferred triples before the query runs.

Querying the inferred graph with Apache Jena Fuseki

We import the inferred graph, including reasoning results and entity links, into Fuseki to run SPARQL queries.

The screenshot shows the Apache Jena Fuseki web interface. At the top, there's a navigation bar with the Apache Jena Fuseki logo, links for datasets, manage, and help, and a server status indicator. Below this, the path `/movies` is shown. A toolbar contains links for query, add data, edit, and info. The main section is titled "SPARQL Query" and includes instructions to enter a query. There are tabs for "Selection of triples" and "Selection of classes". Below these are fields for "SPARQL Endpoint" (containing `/movies/sparql`), "Content Type (SELECT)" (set to JSON), and "Content Type (GRAPH)" (set to Turtle). A "Prefixes" section shows `rdf`, `rdfs`, `owl`, and `xsd`. A SPARQL query is entered in the text area, and a "Run" button is visible. Below the query area, the results are displayed in a table view. The table has columns for director, name, movie, and title. The results show five entries for Andrew Adamson, all linked to "The Chronicles of Narnia: Prince Caspian".

Example Queries

Selection of triples Selection of classes

SPARQL Endpoint: `/movies/sparql` Content Type (SELECT): JSON Content Type (GRAPH): Turtle

Prefixes: `rdf` `rdfs` `owl` `xsd`

```
1 # Directors and their movies, using the inferred inverse relationship.
2 PREFIX kg: <https://example.org/ontology/>
3 SELECT ?director ?name ?movie ?title WHERE {
4   ?director kg:directedMovie ?movie ; kg:name ?name .
5   ?movie kg:name ?title .
6 }
7 ORDER BY ?name ?title
8 LIMIT 20
9
```

Table Response Geo 20 results in 0.011 seconds

director	name	movie	title
1<https://example.org/person/5524>	Andrew Adamson	<http://dbpedia.org/resource/The_Chronicles_of_Narnia_Pr...>	The Chronicles of Narnia: Prince Caspian
2<https://example.org/person/5524>	Andrew Adamson	<http://www.wikidata.org/entity/Q222023>	The Chronicles of Narnia: Prince Caspian
3<https://example.org/person/5524>	Andrew Adamson	<https://example.org/movie/2454>	The Chronicles of Narnia: Prince Caspian
4<https://example.org/person/5524>	Andrew Adamson	<http://dbpedia.org/resource/The_Chronicles_of_Narnia_T...>	The Chronicles of Narnia: The Lion, the Witch and the Wardr...
5<https://example.org/person/5524>	Andrew Adamson	<http://www.wikidata.org/entity/Q485803>	The Chronicles of Narnia: The Lion, the Witch and the Wardr...

Interface via SPARQL endpoint/terminal to query data

When was Avatar released?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?title ?date WHERE {
  <https://example.org/movie/19995>
    kg:name ?title ;
    kg:datePublished ?date .
}
```

title	date
1Avatar	"2009-12-10"^^<http://www.w3.org/2001/XMLSchema#date>

Showing 1 to 1 of 1 entry

<< < 1 > >>

Interface via SPARQL endpoint/terminal to query data

Who directed Avatar?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?director ?name WHERE {
  <https://example.org/movie/19995>
    kg:director ?director .
  ?director kg:name ?name .
}
```

director	name
1<https://example.org/person/2710>	James Cameron

Showing 1 to 1 of 1 entry

<< < 1 > >>

Interface via SPARQL endpoint/terminal to query data

Do we have any Spider-Man movies?

```
PREFIX kg: <https://example.org/ontology/>
```

```
SELECT ?movie ?title WHERE {  
  ?movie a kg:Movie ; kg:name ?title .  
  FILTER(CONTAINS(LCASE(STR(?title)), "spider"))  
}  
ORDER BY ?title
```

movie	title
1<http://dbpedia.org/resource/Spider-Man_2>	Spider-Man 2
2<http://www.wikidata.org/entity/Q190145>	Spider-Man 2
3<https://example.org/movie/558>	Spider-Man 2
4<http://dbpedia.org/resource/Spider-Man_3>	Spider-Man 3
5<http://www.wikidata.org/entity/Q182212>	Spider-Man 3
6<https://example.org/movie/559>	Spider-Man 3
7<http://dbpedia.org/resource/The_Amazing_Spider-Man_(film)>	The Amazing Spider-Man
8<http://www.wikidata.org/entity/Q229808>	The Amazing Spider-Man
9<https://example.org/movie/1930>	The Amazing Spider-Man
10<http://dbpedia.org/resource/The_Amazing_Spider-Man_2>	The Amazing Spider-Man 2
11<http://www.wikidata.org/entity/Q4254026>	The Amazing Spider-Man 2
12<https://example.org/movie/102382>	The Amazing Spider-Man 2

Showing 1 to 12 of 12 entries

<< < 1 > >>

Interface via SPARQL endpoint/terminal to query data

5 Drama films with the highest average rating?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?movie ?title ?average WHERE {
  ?movie a kg:Movie ;
         kg:name ?title ;
         kg:genre ?genre ;
         kg:ratingValue ?average .
  ?genre kg:name "Drama" .
}
ORDER BY DESC(?average) ?title
LIMIT 5
```

movie	title	average
1< http://dbpedia.org/resource/The_Dark_Knight >	The Dark Knight	"8.2"^^< http://www.w3.org/2001/XMLSchema#decimal >
2< http://www.wikidata.org/entity/Q163872 >	The Dark Knight	"8.2"^^< http://www.w3.org/2001/XMLSchema#decimal >
3< https://example.org/movie/155 >	The Dark Knight	"8.2"^^< http://www.w3.org/2001/XMLSchema#decimal >
4< http://dbpedia.org/resource/Interstellar_(film) >	Interstellar	"8.1"^^< http://www.w3.org/2001/XMLSchema#decimal >
5< http://www.wikidata.org/entity/Q13417189 >	Interstellar	"8.1"^^< http://www.w3.org/2001/XMLSchema#decimal >

Showing 1 to 5 of 5 entries

<< < 1 > >>

Interface via SPARQL endpoint/terminal to query data

Which keywords describe Iron Man 2?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?keywordName WHERE {
  <https://example.org/movie/10138>
    kg:keywords ?keyword .
  ?keyword kg:name ?keywordName .
}
ORDER BY ?keywordName
```

keywordName
1aftercreditsstinger
2based on comic book
3malibu
4marvel cinematic universe
5marvel comic
6revenge
7superhero

Showing 1 to 7 of 7 entries

<< < 1 > >>

References I

- [1] Tim Berners-Lee. Linked data. <https://www.w3.org/DesignIssues/LinkedData.html>, 2006.
- [2] W3C RDF Working Group. Rdf 1.1 concepts and abstract syntax.
<https://www.w3.org/TR/rdf11-concepts/>, 2014. W3C Recommendation.
- [3] W3C SPARQL Working Group. Sparql 1.1 query language.
<https://www.w3.org/TR/sparql11-query/>, 2013. W3C Recommendation.